

**White Paper**

Jan 20, 2026

# **CacheFlow: Code Free and Efficient Synchronization between Redis and Databases**

**Beijing Shenqi Data**



## 1. Introduction to CacheFlow

CacheFlow, developed by Beijing Shenqi Data Ltd., is a middleware system designed to automate data synchronization between databases (such as MySQL) and Redis. In a typical database + Redis architecture, Redis caches data derived from the database. When the database changes, those updates must be synchronized to Redis — especially when Redis contains cached results of complex SQL queries. This synchronization is non-trivial.

CacheFlow is built to solve this problem. Its core capabilities include:

- Automatically synchronizing SQL query results from the database into Redis through configuration alone—eliminating the need for custom synchronization code.
- Capturing and utilizing database change logs to ensure real-time synchronization of cached SQL results in Redis.
- Performing incremental computation to update SQL results—avoiding full re-queries of the database and dramatically reducing database load.

In addition to these core synchronization capabilities, CacheFlow also provides:

- A database proxy service with connection pooling and connection sharing to prevent database overload and crashes.
- A built-in SQL optimization and computation engine that enables federated caching across heterogeneous data sources.

### About Beijing Shenqi Data

Founded in 2024, Beijing Shenqi Data specializes in database kernel and related technology R&D. It has developed a data processing core—DB3Kernel—on top of which various database-related applications are built.

### 1.1 Example Scenario: Synchronizing Redis and Databases

Consider an order-processing system with a user table and an order table stored in a database such as MySQL. For performance and load-reduction reasons, user information and aggregated order statistics are cached in Redis. This introduces a challenge: ensuring that Redis remains consistent with MySQL.

CacheFlow automates this synchronization.

## 1.2 Configuration-Based Mapping between Redis Keys and SQL (No Sync Code Required)

In this architecture, Redis acts as a cache while the database is the source of truth. Traditionally:

1. The application queries the database.
2. It writes results into Redis for fast access.
3. When data changes, it must update both the database and Redis.

This approach requires developers to write significant custom code to maintain cache-database consistency—difficult to maintain and error-prone. It becomes especially complex when upgrading database versions (e.g., MySQL 8.1 → 8.2) or switching engines entirely (MySQL → PostgreSQL).

CacheFlow removes this burden. Users simply configure SQL definitions that describe how Redis keys map to database content, and CacheFlow automatically synchronizes the newest database content into Redis.

Example:

| Redis Key      | Value as SQL result                        |
|----------------|--------------------------------------------|
| User:\$1       | Select * from users where id = \$1         |
| CityOrderTotal | Select count(*) from orders group by city; |

CacheFlow supports standard SQL operators, including aggregates, joins (inner and outer), and more.

## 1.3 Real-Time SQL Result Synchronization with Reduced Database Load

CacheFlow does not rely on application business logic. Instead, it listens to database change logs (MySQL binlog, PostgreSQL WAL, etc.) and captures change events in real time. Whenever a row changes:

- CacheFlow analyzes whether the change affects any SQL mapped to Redis keys.
- If so, it proactively updates the corresponding Redis entries—without waiting for cache expiration.

Key benefits:

- Redis cached data is updated in real time. In tests involving single-table and join queries, CacheFlow provides millisecond-level synchronization.

- Because updates are pushed to Redis (not pulled via read-through), applications do not need to frequently query the database to fetch fresh results—substantially reducing database load.

CacheFlow incrementally updates SQL query results in Redis. In user testing, a LEFT JOIN re-queried every second consumed 38% CPU; under CacheFlow, CPU usage was negligible.

## 1.4 Preventing Database Crashes Caused by Cache Breakdown

During cache breakdown events, applications may issue a sudden surge of database queries, potentially causing the database to crash.

To mitigate this, CacheFlow can operate as a DB Proxy, forwarding SELECT statements to the database while employing connection pooling and caching to reduce load. To use CacheFlow as a proxy, applications simply point the database IP to the CacheFlow address.

## 1.5 Federated Caching for Heterogeneous Data Sources

In multi-source or user-sharded environments, users can define multiple data sources and use standard SQL to perform cross-source result caching. This is made possible by CacheFlow's built-in SQL execution engine and deep log-parsing capability across different sources.

Example:

| Config Name | Actual Data Source                                |
|-------------|---------------------------------------------------|
| myusers     | Host: 192.168.1.15, port: 3306, table: users      |
| pglist      | Host: 192.168.1.68, port: 5432, table: forbidlist |

A Redis key may be defined such that MySQL users are filtered by a PostgreSQL forbidlist:

| Redis Key     | Value as SQL result                                                         |
|---------------|-----------------------------------------------------------------------------|
| ValidUser:\$1 | Select * from myusers where id = \$1 and id not in (select id from pglist); |

## 1.6 Additional Features

### Redis Sentinel Mode

CacheFlow supports Sentinel.

### Data Security

CacheFlow requires database read permissions and Redis write permissions. Aside from monitoring, it exposes no external services.

### High Availability

High availability is provided through Consul or user-defined HA frameworks.

## 1.7 Summary: CacheFlow vs. Traditional Cache Synchronization

| Method           | Dev Efforts                                                                   | Complexity               | DB Upgrade  | Heterogeneous DB | Real-time | DB Overhead |
|------------------|-------------------------------------------------------------------------------|--------------------------|-------------|------------------|-----------|-------------|
| Coding (1)       | High: polling DB for changes and send full SQL query to refresh periodically. | Complicated              | Hard        | Hard             | No        | High        |
| Coding (2)       | High: Trigger or CDC monitoring: send full SQL every time                     | Complicated              | Hard        | Hard             | Near      | Medium      |
| Coding (3)       | High: change Redis and DB values at the same time                             | Complicated Transactions | Hard        | Hard             | Almost    | None        |
| <b>CacheFlow</b> | Easy: Config Redis and related SQL pair                                       | None                     | Config Only | Config Only      | Near      | Low         |

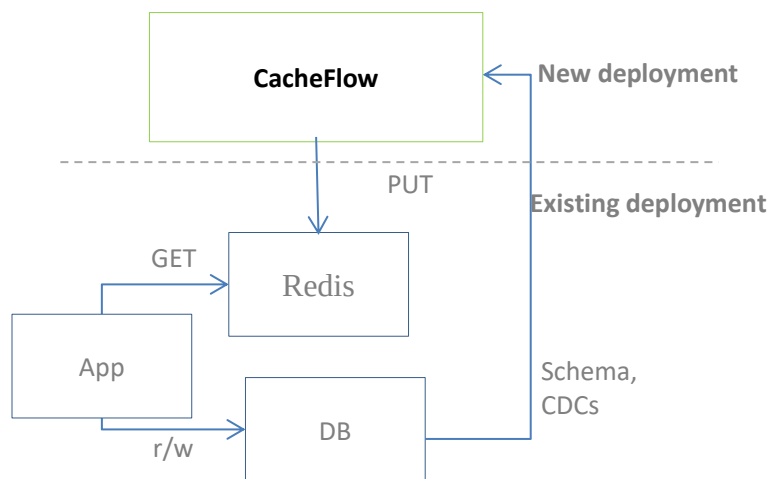
CacheFlow allows Redis content to be defined via SQL—providing timely, efficient, code-free synchronization between Redis and the database.

## 2. Deployment and Use

CacheFlow reads a configuration file to perform automatic cache updates. The configuration file includes:

1. **Database and Redis access settings**—so CacheFlow knows which sources to synchronize. Many-to-many mappings between DB clusters and Redis clusters are supported.
2. **Redis key ↔ SQL mapping definitions**—including support for SQL parameters (\$1, \$2, etc.).

The diagram shows how CacheFlow integrates with existing deployments. No changes are required to application architecture. Applications always see up-to-date Redis data, without custom maintenance code.



## 2.1 System Requirement

CacheFlow 系统运行需求如下:

| Item                                         | Details                                      |
|----------------------------------------------|----------------------------------------------|
| OS Linux or Linux-capable Docker environment | OS Linux or Linux-capable Docker environment |
| CPU x86-64 with AVX2 support                 | CPU x86-64 with AVX2 support                 |
| Cache Redis 5.0+                             | Cache Redis 5.0+                             |
| Databases MySQL 8.0/8.1; PostgreSQL 15+      | Databases MySQL 8.0/8.1; PostgreSQL 15+      |

## 2.2 One-Click Trial

On Linux, users can start a full demo environment via:

```
curl -fsSL https://gitlab.com/korenhe/cf3demo/-/raw/main/start.sh | bash
```

This command downloads and launches images for CacheFlow, MySQL, Redis, and a sample application my-test that simulates an order system. The system updates MySQL continuously and reads from Redis—you will observe that Redis always reflects real-time MySQL results.

## 2.3 System Monitoring

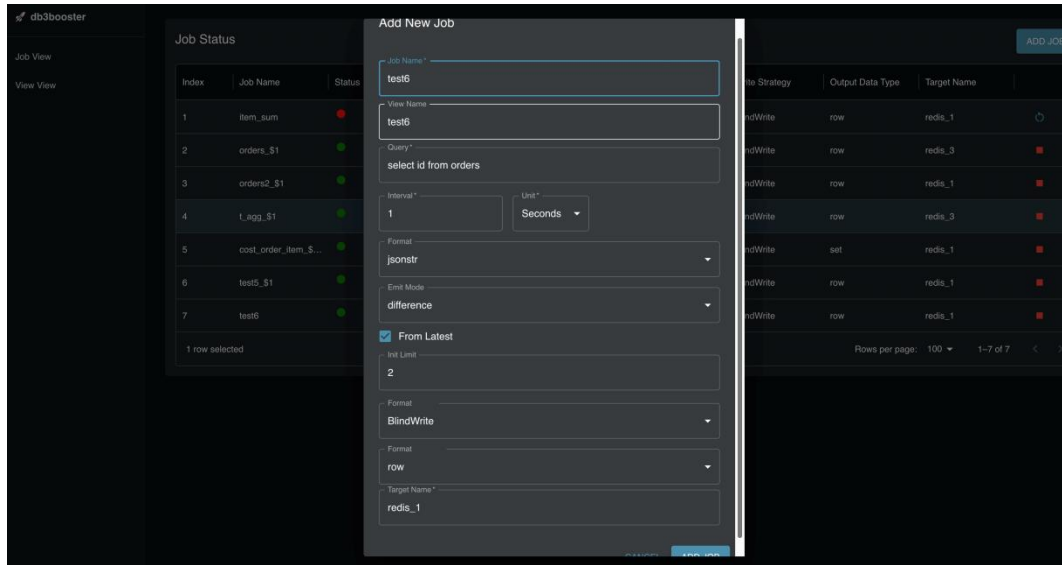
CacheFlow provides both RESTful APIs and a GUI dashboard for monitoring.

This example page show current status of all synchronization tasks.

The screenshot shows the 'db3booster' Job Status dashboard. It features a table with columns: Index, Job Name, Status, Interval, Format, Emit Mode, From Latest, Init Limit, Write Strategy, Output Data Type, and Target Name. There are 7 rows of data. The 'Status' column uses colored dots (red for error, green for success). The 'From Latest' column has checkboxes. The 'Init Limit' column has numerical values. The 'Write Strategy' column has 'BlindWrite'. The 'Output Data Type' column has 'row' or 'set'. The 'Target Name' column has 'redis\_1' or 'redis\_3'. A '1 row selected' message is at the bottom left, and 'Rows per page: 100' and '1-7 of 7' are at the bottom right.

| Index | Job Name              | Status | Interval | Format  | Emit Mode  | From Latest                         | Init Limit | Write Strategy | Output Data Type | Target Name |
|-------|-----------------------|--------|----------|---------|------------|-------------------------------------|------------|----------------|------------------|-------------|
| 1     | item_sum              | ●      | 1s       | jsonstr | difference | <input type="checkbox"/>            | -1         | BlindWrite     | row              | redis_1     |
| 2     | orders_\$1            | ●      | 1s       | jsonstr | difference | <input type="checkbox"/>            | -1         | BlindWrite     | row              | redis_3     |
| 3     | orders2_\$1           | ●      | 1s       | jsonstr | difference | <input type="checkbox"/>            | -1         | BlindWrite     | row              | redis_1     |
| 4     | t_agg_\$1             | ●      | 1s       | jsonstr | difference | <input type="checkbox"/>            | -1         | BlindWrite     | row              | redis_3     |
| 5     | cost_order_item_\$... | ●      | 1s       | jsonstr | difference | <input type="checkbox"/>            | -1         | BlindWrite     | set              | redis_1     |
| 6     | test5_\$1             | ●      | 1s       | jsonstr | difference | <input type="checkbox"/>            | -1         | BlindWrite     | row              | redis_1     |
| 7     | test6                 | ●      | 1s       | jsonstr | difference | <input checked="" type="checkbox"/> | 2          | BlindWrite     | row              | redis_1     |

This page is adding tasks dynamically through the GUI.



CacheFlow provides full monitoring and management capabilities.

## 2.4 High-Availability Deployment

CacheFlow supports HA via Consul or can integrate with user HA frameworks.

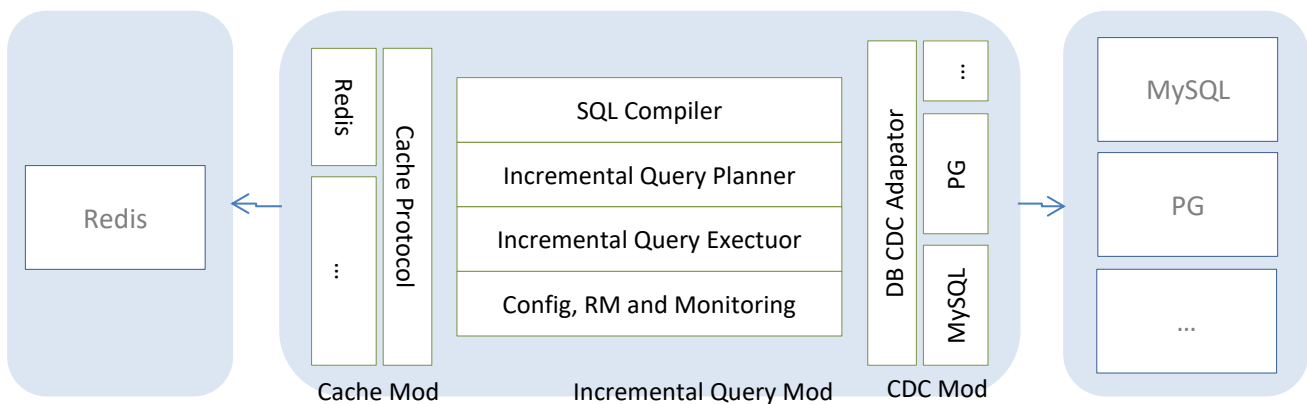
## 2.5 Developer Documentation

Latest documentation:

<https://gitlab.com/cf36755403/cf3/-/tree/main/2.0.6>

## 3. CacheFlow Architecture

The following diagram illustrates the foundational components of CacheFlow:



## Components:

1. Database Change Capture Module
  - Reads logs (binlog, logical logs, etc.)
  - Converts all changes (insert/update/delete/DDL) into a unified format
2. Incremental Computation Module
  - Determines which SQL statements are affected by each change event
  - Performs incremental computation of affected SQL queries
  - Handles DDL by restarting or disabling related SQL definitions
  - Supports redo-based fault recovery
3. Cache System Module
  - Writes key-value updates into Redis
  - Persists SQL computation results into corresponding Redis keys
4. Administration & Monitoring Module
  - Manages data sources, caches, and SQL-to-key mappings
  - Provides RESTful APIs for adding, deleting, starting, and stopping tasks

## 4. User Case Studies



CacheFlow验证报告  
1.pdf